

# SIMATS ENGINEERING ASSESSMENT TOOLS

**COURSE NAME:** Theory of Computation **COURSE CODE:** CSA1304

## **COURSE OUTCOME COVERED**

**CO5:** *Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques.*  
( BL: 3)

*Assessment Tool Weightage: 20%*

**QUESTIONS: 5 Total Marks:25 Duration : 1 Hour**  
**Design Critique Session**

### **Code of Conduct:**

*I Yuvann Nithish R (192411267)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student: Yuvann Nithish R**

# SIMATS ENGINEERING ASSESSMENT TOOLS

- 1 A PDA is designed to recognize  $a^n b^n$ , but it fails for  $a^n b^n c^n$ . Critique the design and identify possible flaws in **stack operations** and **state transitions**. How would you improve it?

- ## 2 A student claims that **Context-Free Grammars (CFGs)**

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T^* F \mid F$$
$$F \rightarrow (E) \mid \text{id}$$

**can be directly converted into a Deterministic PDA**

**(DPDA).**Critically evaluate this statement with justification and suggest corrections if needed

- 3 A Turing Machine is designed using **multiple tracks** to solve a problem. Suggest alternative **programming techniques** like checking-off symbols or subroutines to do subtraction

- #### 4 While designing a PDA accepts the language

$$\begin{aligned} \mathbb{Q}^{\otimes 2} &= \{ \mathbb{Q}^{\otimes 2} \mid \mathbb{Q}^{\otimes 2} \in (\mathbb{Q}^{\otimes 2} + \mathbb{Q}^{\otimes 2}) * \mathbb{Q}^{\otimes 2} \otimes \mathbb{Q}^{\otimes 2} \otimes \mathbb{Q}^{\otimes 2} \otimes \mathbb{Q}^{\otimes 2} \\ &\quad \mathbb{Q}^{\otimes 2} \otimes (\mathbb{Q}^{\otimes 2}) = \mathbb{Q}^{\otimes 2} \otimes (\mathbb{Q}^{\otimes 2}) \} \end{aligned}$$

 $\diamond\diamond\dots(\diamond\diamond)$  is the number of a's in w

◆◆◆..(◆◆) is the number of b's in w, acceptance is defined only by **final state**, ignoring empty stack acceptance. Critique this design choice. Would it affect the language recognized? Explain with reasoning.

- 5 A system designer suggests replacing a Turing Machine with a PDA to reduce computational complexity. Critically compare FA, PDA, and Turing Machine capabilities and evaluate whether this replacement is valid.**

# 1. PDA designed for $(a^n b^n)$ fails for $(a^n b^n c^n)$

A PDA for

[  
 $L = \{a^n b^n \mid n \geq 1\}$   
]

normally uses the stack to remember the number of  $a$ s.

Typical working

- 

For every  $a$ , push one stack symbol ( $A$ ).

- 
- 

For every  $b$ , pop one ( $A$ ).

- 
- 

Accept when all ( $A$ )'s are removed.

- 

For example:

[  
aaabbb  
]

gives:

$a \rightarrow \text{push } A$   
 $a \rightarrow \text{push } A$   
 $a \rightarrow \text{push } A$

$b \rightarrow \text{pop } A$   
 $b \rightarrow \text{pop } A$   
 $b \rightarrow \text{pop } A$

This works because the PDA needs to compare only **two quantities**.

Why it fails for  $(a^n b^n c^n)$

For

[  
 $L = \{a^n b^n c^n \mid n \geq 1\}$   
]

the PDA has to compare **three quantities**:

[  
 $\#a = \#b = \#c$   
]

A single stack can easily compare the number of  $a$ s with the number of  $b$ s, but after popping the stack during the  $b$  phase, the information about the number of  $a$ s has been lost.

For example:

aaabbbccc

after  $a$ 's: AAA  
after  $b$ 's: empty

When the machine starts reading  $c$ , there is no longer any stack information telling it that there were exactly three  $a$ s.

## Possible flaws

### 1. Incorrect stack operation

The PDA may pop all  $A$ s while processing  $b$ s and then have nothing left to compare with the  $c$ s.

### 2. Incorrect state transition

The transition from the  $b$  state to the  $c$  state may occur without verifying that the number of  $b$ s exactly matched the number of  $a$ s.

### 3. No mechanism for three-way comparison

A single ordinary PDA cannot maintain two independent counters in the required way.

## Improvement

The important conclusion is:

[  
 $\boxed{\{a^n b^n c^n\}}$  is not a context-free language  
]

Therefore, an ordinary PDA **cannot be improved to recognize this language exactly**.

The appropriate machine is a **Turing Machine**, which can mark symbols and repeatedly compare all three groups.

For example:

```
aabbcc
↓
X a Y b Z c
↓
X X Y Y Z Z
↓
ACCEPT
```

## Conclusion

The PDA design is not merely suffering from a small transition error. The fundamental problem is that:

[  
 $\boxed{\text{PDA is insufficient for } \{a^n b^n c^n\}}$   
]

A TM should be used instead.

---

## 2. Can the given CFG be directly converted to a DPDA?

Given:

[  
 $E \rightarrow E+T \mid T$   
]

[  
 $T \rightarrow T * F \mid F$   
]

[  
 $F \rightarrow (E) \mid id$   
]

The student's statement is:

"A CFG can be directly converted into a DPDA."

## Evaluation

The statement is **not completely correct**.

A general CFG can be converted to a PDA, but the standard CFG-to-PDA construction produces a **Nondeterministic PDA (NPDA)**.

It does **not automatically produce a DPDA**.

Why?

Consider:

$$\begin{array}{l} [ \\ E \rightarrow E+T \mid T \\ ] \end{array}$$

When the PDA has (E) on the stack, it may have to choose between two productions:

$$\begin{array}{l} [ \\ E \rightarrow E+T \\ ] \end{array}$$

or

$$\begin{array}{l} [ \\ E \rightarrow T \\ ] \end{array}$$

A general CFG does not provide enough information for a deterministic PDA to always make the correct choice.

Another problem: left recursion

The grammar contains:

$$\begin{array}{l} [ \\ E \rightarrow E+T \\ ] \end{array}$$

and

$$\begin{array}{l} [ \\ T \rightarrow T * F \\ ] \end{array}$$

These are **left-recursive** productions.

This grammar is therefore not directly suitable for a simple predictive/LL(1) DPDA construction.

---

## Can this language be recognized by a DPDA?

Yes.

The language represented by this expression grammar is deterministic and can be parsed using deterministic parsing techniques such as:

- 

LL(1), after suitable grammar transformation

- 
- 

LR parsing

- 
- 

Shift-reduce parsing

- 

So the problem is not that the language cannot be recognized by a DPDA.

The problem is the student's claim that it can be **directly converted**.

---

## Correct the grammar

Remove left recursion:

```
[  
E → TE'  
]
```

```
[  
E' → +TE' | ε  
]
```

```
[  
T → FT'  
]
```

$$[ T \rightarrow *FT \mid \epsilon ]$$

$$[ F \rightarrow (E) \mid id ]$$

This form is much more suitable for deterministic predictive parsing.

### Example

For:

$$[ id + id * id ]$$

the parser can deterministically identify:

$$\begin{array}{c} id \\ + \\ id \\ * \\ id \end{array}$$

while respecting the grammar's precedence structure.

### Final conclusion

The student's statement should be corrected to:

A general CFG can be converted directly to an NPDA, but not necessarily to a DPDA. This particular expression language is deterministic, so a DPDA can be constructed after suitable grammar transformation or by using deterministic parsing techniques.

Therefore:

$$[ \boxed{\text{CFG} \rightarrow \text{NPDA always}} ]$$

but:

$$[ \boxed{\text{CFG} \not\rightarrow \text{DPDA directly in general}} ]$$

---

### 3. Alternative TM programming techniques to multiple tracks

Multiple tracks are one way to organize information on a Turing Machine tape.

However, they are **not necessary**. The same computation can often be implemented using ordinary tape symbols and systematic programming techniques.

#### A. Checking-off symbols

This is one of the most useful techniques.

Suppose we want to subtract two unary numbers:

```
[  
11111-111  
]
```

We can mark symbols as they are used.

For example:

```
11111#111
```

Mark one symbol from the first number and one from the second:

```
x1111#x11
```

Repeat:

```
xx111#xx1
```

Repeat again:

```
xxx11#xxx
```

The remaining symbols on the first side are:

```
xx
```

Therefore:

```
[  
5-3=2  
]
```

## Advantage

The machine does not need a second track. The tape symbols themselves store the information.

---

## B. Subroutines

A complex TM can be divided into reusable operations.

For example:

### SUBROUTINE 1

Move to the left end.

LEFT

### SUBROUTINE 2

Find the next unmarked symbol.

FIND

### SUBROUTINE 3

Mark a symbol.

MARK

### SUBROUTINE 4

Return to the beginning.

RETURN

The complete machine can call these operations repeatedly.

This makes the TM easier to design and debug.

---

## C. Marking symbols

Instead of multiple tracks, use additional tape symbols.

For example:

```
[
0\rightarrow X
]
```

```
[
1\rightarrow Y
]
```

Here  $x$  and  $y$  remember which symbols have already been processed.

---

## D. Delimiters

A special symbol can separate different pieces of information.

For example:

```
11111#111
```

where  $\#$  separates the two numbers.

This is especially useful when solving arithmetic problems.

---

## E. State-based memory

Small amounts of information can be stored directly in the finite control.

For example, the state can remember:

```
q_found_0
q_found_1
q_carry
q_borrow
```

This eliminates the need for an additional tape track.

## Conclusion

Multiple tracks are mainly a **programming convenience**, not an increase in the fundamental computational power of a TM.

```
[
\boxed{\text{Checking-off + markers + subroutines + states can replace multiple
tracks}}
]
```

---

## 4. PDA for equal numbers of (a)'s and (b)'s — final-state acceptance

The intended language is:

[  
L = {w | w ∈ (a+b)\*, ; count\_a(w) = count\_b(w)}  
]

That means the string may contain a and b in **any order**, as long as:

[  
#a = #b  
]

Examples:

[  
ε, quad ab, quad ba, quad aabb, quad abab, quad abba  
]

are accepted.

---

## Is final-state acceptance a problem?

No.

A PDA can be defined using either:

1.

acceptance by empty stack, or

2.

3.

acceptance by final state.

4.

For nondeterministic PDAs, these two acceptance methods are **equivalent in expressive power**.

Therefore:

```
[  
\boxed{\text{Using final-state acceptance does not reduce the class of CFLs  
recognized.}}  
]
```

---

## But there can be a design flaw

The problem occurs if the PDA was originally designed assuming:

“The stack must become empty for acceptance”

and then the designer simply changes the acceptance condition to:

“Reach a final state.”

That can change the behavior of **that particular PDA**.

For example, suppose the PDA reaches:

```
qf  
stack = AAZ0
```

If it accepts merely because it entered ( $q_f$ ), it may accept a string that should not be accepted.

Therefore, the transition to the final state must be carefully designed.

---

## A better PDA design

For equal numbers of  $a$  and  $b$  in arbitrary order, the PDA can use the stack as a cancellation mechanism.

When reading  $a$

If the stack contains an unmatched  $b$ , pop it.

Otherwise, push an  $A$ .

Conceptually:

```

input a + top B → pop B
input a + top Z0 → push A
input a + top A → push A

```

When reading **b**

If the stack contains an unmatched **a**, pop it.

Otherwise, push **a B**.

Conceptually:

```

input b + top A → pop A
input b + top Z0 → push B
input b + top B → push B

```

At the end, if all symbols have been cancelled, the stack is back to (**Z<sub>0</sub>**).

Then use an epsilon transition:

```

[
(q,\epsilon,Z_0)\rightarrow(q_f,Z_0)
]

```

Thus acceptance is by **final state**.

**Example:** **aabb**

```

a → push A
a → push A
b → pop A
b → pop A

```

Stack returns to:

```

[
Z_0
]

```

Then:

```

[
q\rightarrow q_f
]

```

So:

[  
\boxed{aabb\text{ is accepted}}  
]

For:

aab

one  $A$  remains.

Therefore:

[  
\boxed{aab\text{ is rejected}}  
]

## Conclusion

Using final-state acceptance is perfectly valid.

The important point is:

[  
\boxed{\text{Final-state and empty-stack acceptance are equivalent for NPDA power}}  
]

but the actual transition design must ensure that the final state is reached **only when the stack represents a completely balanced string**.

---

# 5. FA vs PDA vs Turing Machine

A system designer proposes:

“Replace a Turing Machine with a PDA to reduce computational complexity.”

This is **not generally valid**.

The three machines have different computational capabilities.

---

## 1. Finite Automaton

A Finite Automaton has:

- 
- finite states
- 
- 
- no stack
- 
- 
- no unlimited memory
- 

It recognizes:

[  
 $\boxed{\text{Regular Languages}}$   
 ]

Examples:

[  
 $L = \{w \mid w \text{ contains an even number of } 0\text{'s}\}$   
 ]

Regular expressions can describe such languages.

### Limitation

An FA cannot recognize:

[  
 $\{a^n b^n \mid n \geq 0\}$   
 ]

because it cannot remember an arbitrary number of a's.

## 2. Pushdown Automaton

A PDA has:

- 
- finite states

•  
•

an input tape

•  
•

one unbounded stack

•

It recognizes:

[  
\boxed{\text{Context-Free Languages}}  
]

Example:

[  
 $L = \{a^n b^n \mid n \geq 0\}$   
]

The stack can remember the number of as.

**Limitation**

A standard PDA cannot recognize:

[  
\boxed{\{a^n b^n c^n \mid n \geq 1\}}  
]

because it needs to compare three quantities.

---

## 3. Turing Machine

A Turing Machine has:

•

finite states

•  
•

an unbounded tape

- 
- 

a read/write head

- 
- 

ability to move left and right

- 

It can recognize a much larger class of languages.

For example:

[  
 $\{a^n b^n c^n\}$   
]

can be recognized by a TM.

It can also perform general algorithms such as:

- 

arithmetic

- 
- 

sorting

- 
- 

simulation

- 
- 

parsing

- 
- 

searching

- 
-

computation

•

---

## Comparison

| Feature             | FA                 | PDA           | TM                     |
|---------------------|--------------------|---------------|------------------------|
| Memory              | Finite states      | Stack         | Unbounded tape         |
| Storage             | No external memory | LIFO stack    | Read/write tape        |
| Movement            | Input moves right  | Input + stack | Left and right         |
| Language class      | Regular            | Context-free  | Recursively enumerable |
| $(a^n b^n)$         | ✗                  | ✓             | ✓                      |
| $(a^n b^n c^n)$     | ✗                  | ✗             | ✓                      |
| General computation | ✗                  | Limited       | ✓                      |
| Power               | Lowest             | Medium        | Highest                |

The hierarchy is:

[  
\boxed{FA < PDA < TM}  
]

in computational power.

---

## Can a TM always be replaced by a PDA?

No.

Suppose the TM is solving:

[  
 $L = \{a^n b^n c^n \mid n \geq 1\}$   
]

A PDA cannot recognize this language.

Therefore replacing the TM with a PDA would make the system **unable to solve the original problem**.

When is replacement valid?

A TM may be replaced by a PDA only if the particular problem is known to be context-free and does not require the additional computational power of a TM.

For example:

[  
 $\{a^n b^n\}$   
]

can be handled by a PDA.

But:

[  
 $\{a^n b^n c^n\}$   
]

cannot.

**Final conclusion**

The designer's proposal is valid **only for problems that fall within the CFL class**.

It is not a general optimization strategy.

[  
 $\boxed{\text{Reducing machine power may reduce complexity, but it can also destroy correctness.}}$   
]

The correct approach is to first determine the computational requirements of the language/problem and then choose the weakest machine capable of solving it.

## SIMATS ENGINEERING ASSESSMENT TOOLS

### RUBRICS FOR CALCULATION

| Criteria                                 | Weight<br>age | Level 4 –<br>Exemplary<br>(5<br>Marks)                                       | Level 3 –<br>Proficient<br>(4 Marks)            | Level 2 –<br>Developing<br>(2–3 Marks)                    | Level 1 –<br>Beginning<br>(0– 1 Mark) |
|------------------------------------------|---------------|------------------------------------------------------------------------------|-------------------------------------------------|-----------------------------------------------------------|---------------------------------------|
| <b>Conceptual<br/>Understandin<br/>g</b> | <b>30</b>     | Demonstrates<br>clear and<br>complete<br>understand<br>ing of the<br>concept | Good<br>understanding<br>with minor<br>gaps     | Basic<br>understanding<br>with some<br>misconception<br>s | Poor or<br>incorrect<br>understanding |
| <b>Accuracy<br/>of Answer</b>            | <b>25</b>     | Completely<br>correct answer<br>with<br>no errors                            | Mostly<br>correct with<br>minor<br>errors       | Partially<br>correct with<br>noticeable<br>errors         | Incorrect or<br>irrelevant<br>answer  |
| <b>Application<br/>of Knowledge</b>      | <b>20</b>     | Correctly<br>applies concepts<br>to<br>solve the<br>question/problem         | Applies<br>concepts<br>with minor<br>mistakes   | Limited or<br>partially correct<br>application            | No or<br>incorrect application        |
| <b>Completeness of<br/>Response</b>      | <b>15</b>     | Covers all<br>required parts<br>of the<br>question                           | Covers<br>most parts<br>with<br>minor omissions | Covers only<br>some parts                                 | Incomplete<br>or missing<br>response  |
| <b>Clarity &amp;<br/>Presentation</b>    | <b>10</b>     | Clear,<br>concise, and<br>well-<br>organized answer                          | Understandable<br>with minor<br>issues          | Somewhat<br>unclear or<br>poorly<br>structured            | Difficult to<br>understand            |